Robby Emacs Configuration

Robby L. Johnson III Dasker Joe Teahen

October 12, 2026

Contents

1 Split Custom Files **3**

2 Package Management Setup **3**
2.1 `use-package` Setup 3

3 Startup Stuff **4**
3.1 Keep Folders Clean 4
3.2 Auto updates 4

4 Emacs Utilities **5**
4.1 Which Key Package 5
4.2 Ace Window Manager 5
4.3 Helpful Package 5

5 UI Improvements **5**
5.1 Emacs Startup and Basic UI 5
5.2 Line and Column UI 6
5.3 Color Theme `modus-themes` 6
5.4 Rainbow Delimiters Package 6
5.5 Modeline 7
5.5.1 Doom Mode Line 7
5.6 Spacious Padding 7

6 Editing Improvements **7**
6.1 Tabs 7
6.2 Tool Tip Injections 8
6.3 Electric Pairs 8
6.4 Window Open Behavior 8

6.5 Spell Check . . . 8
6.6 Template System (`yasnippet`) . . . 9

7 Keybindings **9**
7.1 Becoming Evil . . . 9
7.2 Visual Adjustments . . . 10
7.3 Default Keybindings . . . 10
7.3.1 New Tab Buffer . . . 10
7.3.2 ESC Prompts . . . 10

8 Minibuffer/Buffer Completions **11**
8.1 Vertico . . . 11
8.2 Marginalia . . . 11
8.3 Orderless . . . 12
8.4 Consult . . . 12
8.5 Embark . . . 12
8.6 Corfu . . . 13

9 Org Mode **13**
9.1 Org Faces . . . 13
9.2 Basic Setup . . . 14
9.3 Org Auto Tangle . . . 17
9.4 Evil-Org . . . 17
9.5 Citations . . . 17
9.5.1 `citar` . . . 17
9.6 Visual Column Alignment . . . 18
9.7 Org Modern . . . 19
9.8 Org Tempo . . . 19
9.9 Org Code Listings . . . 19
9.10 Org Babel Languages Configuration . . . 20

10 Terminals and Shells **20**
10.1 vterm . . . 20
10.2 Vscode Terminal Buffer . . . 20
10.2.1 Function . . . 20
10.2.2 Keybind . . . 21

1 Split Custom Files

This bit of code splits all emacs writing of custom variables to a different file. This just keeps my `init.el` file looking like I expect.

```
                                        ; When you do automatic config
↪  changing, put the changes in custom.el
                                        ; instead of this init file
                                        ; Change
  (setq custom-file
        (expand-file-name "custom.el" user-emacs-directory))
  (load custom-file)
```

2 Package Management Setup

```
                                        ; Loads the default package
↪  manager
  (require 'package)

                                        ; Loads 3 different package
                                        ↪  archives
  (setq package-archives
        '(("melpa" . "https://melpa.org/packages/")
          ("org" . "https://orgmode.org/elpa/")
          ("elpa" . "https://elpa.gnu.org/packages/")))

                                        ; Essentially APT Update
  (package-initialize)
  (unless package-archive-contents
    (package-refresh-contents))

  ;; Auto tangle org files like emacs.org
  (use-package org-auto-tangle
    :defer t
    :hook (org-src-mode . org-auto-tangle-mode))
```

2.1 use-package Setup

```
                                        ; Install the use-package
↪  package manager
  (require 'use-package)
                                        ; Update some use-package
                                        ↪  options
                                        ; Set default to be ensure
                                        ↪  instead of no-ensure
```

```
                                        ; AKA don't be lazy unless I say
                                        ↪  so
(setq use-package-always-ensure t)       ; Turns ensure on by default
(setq use-package-compute-statistics t)  ; Turns on startup statistics
```

3 Startup Stuff

3.1 Keep Folders Clean

```
;; moves backup files to .emacs.d/tmp/backups
(setq backup-directory-alist
      `(("." .
         ,(expand-file-name "tmp/backups/" user-emacs-directory))))

;; auto-saves
(make-directory (expand-file-name "tmp/auto-saves/"
↪  user-emacs-directory) t)
(setq auto-save-list-file-prefix
      (expand-file-name "tmp/auto-saves/sessions/" user-emacs-directory)
      auto-save-file-name-transforms
      `((".*" ,(expand-file-name "tmp/auto-saves/" user-emacs-directory)
      ↪  t)))

(use-package no-littering)
```

3.2 Auto updates

Sets up auto updates of packages!

```
(use-package auto-package-update
  :demand t
  :custom
  (auto-package-update-interval 7)
  (auto-package-update-prompt-before-update t)
  (auto-package-update-hide-results t)
  :config
  (auto-package-update-maybe)
  (auto-package-update-at-time "09:00"))
```

4 Emacs Utilities

4.1 Which Key Package

```
(use-package which-key
  :defer nil
  :diminish which-key-mode
  :config (which-key-mode)
  :custom
  (which-key-idle-delay 0.5))
```

4.2 Ace Window Manager

```
(use-package ace-window
  :ensure t
  :bind
  ("C-a" . ace-window)
  :custom
  (aw-dispatch-always t)
  (aw-keys '(?a ?s ?d ?f ?g ?h ?j ?k ?l)))
```

4.3 Helpful Package

```
(use-package helpful
  :commands (helpful-callable helpful-variable helpful-command
  ↪  helpful-key))
```

5 UI Improvements

5.1 Emacs Startup and Basic UI

```
(setq inhibit-splash-screen t)    ; No splash screen.
(setq inhibit-startup-message t)  ; No startup message.

(setq frame-title-format "Robby Emacs")

(scroll-bar-mode -1)          ; Disable visual scrolbar
(tool-bar-mode -1)            ; Disable the toolbar
(tooltip-mode -1)             ; Disable tooltips
(fringe-mode '(1 . 1))        ; Sets frame to a minimum of one character
(menu-bar-mode  -1)           ; Disable the menu bar

(setq ring-bell-function 'ignore) ; Turn off bell/screen flash
```

5.2 Line and Column UI

```
(column-number-mode)                ; Adds column number to mode line
(global-display-line-numbers-mode t) ; Adds line numbers
(add-hook 'prog-mode-hook 'display-line-numbers-mode)

(setopt display-fill-column-indicator-column 80)
(add-hook 'prog-mode-hook #'display-fill-column-indicator-mode)

;; Disable line numbers for some modes
(dolist (mode '(org-mode-hook
                vterm-mode-hook
                term-mode-hook
                shell-mode-hook
                eshell-mode-hook))
  (add-hook mode (lambda () (display-line-numbers-mode 0))))
```

5.3 Color Theme `modus-themes`

```
(use-package modus-themes
  :ensure t
  :demand t
  :custom
  (modus-themes-common-palette-overrides
   '((fg-region unspecified)        ;; highlight not change underlying
   ↪  style
     (fg-paren-match (extrabold)))) ;; matching parenthesis are bolded

  ;; changes the highlighting of completions
  (modus-themes-completions '((matches . (bold))
                              (selection . (extrabold italic))))

  ;; adds bold to some syntax elements
  (modus-themes-bold-constructs t)
  (modus-themes-italic-constructs t)
  :config
  (modus-themes-load-theme 'modus-vivendi-deuteranopia))
```

5.4 Rainbow Delimiters Package

Colors paired characters to try to differentiate them to the eye.

```
(use-package rainbow-delimiters
  :defer t
  :hook (prog-mode . rainbow-delimiters-mode))
```

5.5 Modeline

5.5.1 Doom Mode Line

```
(use-package doom-modeline
  :ensure t
  :init (doom-modeline-mode 1)
  :custom
  (doom-modeline-height 25)      ; Increases the overall height of the
  ↪  modeline bar
  (doom-modeline-icon-scale 1.0) ; Scales the icon size relative to the
  ↪  font (try 0.9 or 1.1 if needed)
  (doom-modeline-v-adjust 0.05)) ; Adjusts vertical position to push
  ↪  text away from icons
```

5.6 Spacious Padding

```
(use-package spacious-padding
  :config
  (spacious-padding-mode 1))
```

6 Editing Improvements

6.1 Tabs

Dealing with how tabs are handled in emacs **something I don't like about emacs, this is a first**

```
(setq custom-tab-width 4)
(setq tab-width custom-tab-width)

(defun disable-tabs () (setq indent-tabs-mode nil))
(defun enable-tabs ()
  (local-set-key (kbd "TAB") 'tab-to-tab-stop)
  (setq indent-tabs-mode t)
  (setq tab-width custom-tab-width))

(add-hook 'prog-mode-hook 'disable-tabs)
(add-hook 'makefile-mode-hook 'enable-tabs)

(setq-default electric-indent-inhibit t)
(setq backward-delete-char-untabify-method 'hungry)
```

6.2 Tool Tip Injections

```
(use-package marginalia
  :ensure t
  :bind (:map minibuffer-local-map
              ("M-A" . marginalia-cycle))
  :init
  (marginalia-mode))
```

6.3 Electric Pairs

Automatically closes paired characters when typing them (i.e., {}).

```
  (electric-pair-mode 1)

  ;; Region wraps
  (with-eval-after-load 'evil
    (evil-define-key 'visual 'electric-pair-mode-map
      (kbd "C-{") 'insert-pair)
    (evil-define-key 'visual 'electric-pair-mode-map
      (kbd "C-<") 'insert-pair)
    (evil-define-key 'visual 'electric-pair-mode-map
      (kbd "C-(") 'insert-pair)
    (evil-define-key 'visual 'electric-pair-mode-map
      (kbd "C-\"") 'insert-pair)
    (evil-define-key 'visual 'electric-pair-mode-map
      (kbd "M-[") 'insert-pair))
```

6.4 Window Open Behavior

Stops Emacs from messing with window layouts too much.

```
  ; changes default behavior on what emacs does when displaying windows
  (customize-set-variable 'display-buffer-base-action
                          '((display-buffer-reuse-window
                          ↪  display-buffer-same-window)
                            (reusable-frames . t)))

  (customize-set-variable 'even-window-sizes nil) ; avoid resizing
```

6.5 Spell Check

```
  (use-package flyspell
    :defer t
    :hook
    (text-mode . flyspell-mode)
```

```
  (prog-mode . flyspell-prog-mode)
  :custom
  (ispell-extra-args '("--lang=en_US")))
```

6.6 Template System (yasnippet)

```
(use-package yasnippet
  :ensure t
  :hook ((text-mode
          prog-mode
          conf-mode
          snippet-mode) . yas-minor-mode-on))

;; Sample Snippets
(use-package yasnippet-snippets
  :after yasnippet)
```

7 Keybindings

This is a section related to setting up the ability to use different keybindings. Keybindings for particular packages or behaviors will be located with the code for that package. Default keybindings (i.e., scale text) is located in this section.

7.1 Becoming Evil

```
(use-package evil
  :demand t ; Load ASAP
  :init
  (setq evil-want-integration t)
  (setq evil-want-keybinding nil)
  (setq evil-want-C-u-scroll t)
  (setq evil-want-C-i-jump nil)
  :config
  (evil-mode 1)
  (define-key evil-insert-state-map (kbd "C-g") 'evil-normal-state)

  ;; Use visual line motions even outside of visual-line-mode buffers
  (evil-global-set-key 'motion "j" 'evil-next-visual-line)
  (evil-global-set-key 'motion "k" 'evil-previous-visual-line)

                                        ; Start in normal mode.
  (evil-set-initial-state 'messages-buffer-mode 'normal)
```

```
  (evil-set-initial-state 'dashboard-mode 'normal))

(use-package evil-collection
  :after evil
  :config
  ;; Makes sure I can use RET key to submit commands in modes like
  ↪  eshell
  ;; and vterm, etc.
  (setq evil-collection-repl-submit-state 'insert)
  (evil-collection-init))

;; Automatically tangle our emacs.org config file when you save it
(defun dasker/org-babel-tangle-config ()
  (when (string-equal (buffer-file-name)
                      (expand-file-name "~/.emacs.d/emacs.org"))
    ;; Dyanamic scoping to the resuce
    (let ((org-confirm-babel-evaluate nil))
      (org-babel-tangle))))

;; Set hook to try to run above function on orgmode save
(add-hook 'org-mode-hook
          (lambda ()
            (add-hook 'after-save-hook
                      #'dasker/org-babel-tangle-config)))
```

7.2 Visual Adjustments

Set keybinds for text sizing

```
  (global-set-key (kbd "C-+") 'text-scale-increase)
  (global-set-key (kbd "C--") 'text-scale-decrease)
```

7.3 Default Keybindings

7.3.1 New Tab Buffer

C-t opens a new tab buffer

```
  (global-set-key (kbd "C-t") 'tab-bar-new-tab)
```

7.3.2 ESC Prompts

ESC quits prompts

```
(global-set-key (kbd "<escape>") 'keyboard-escape-quit)
```

8 Minibuffer/Buffer Completions

This is a section to handle the minibuffer completion framework for my emacs

8.1 Vertico

Base minibuffer completions frontend (works with the emacs backend).

```
(use-package vertico
  :bind (:map vertico-map
              ("C-j" . 'vertico-next)
              ("C-k" . 'vertico-previous))
  :custom
  (vertico-resize nil)              ;; Grow and shrink the Vertico
  ↪  mini-buffer
  (vertico-cycle t)                 ;; Enable cycling for
  ↪  `vertico-next/previous`
  (context-menu-mode t)
  ;;(enable-recursive-minibuffers t) ;; Allow for minibuffers in
  ↪  existing minibuffers
  :init
  (vertico-mode))

;; Persist history over Emacs restarts.
(use-package savehist
  :init
  (savehist-mode))

;; Make minibuffer appear at center of screen
(use-package vertico-posframe
  :after vertico
  :init
  (vertico-posframe-mode 1))
```

8.2 Marginalia

Adds info to the margins of the vertico completions.

```
(use-package marginalia
  :init
  (marginalia-mode))

(use-package all-the-icons-completion
  :after (marginalia all-the-icons)
  :hook (marginalia-mode . all-the-icons-completion-marginalia-setup)
  :init
```

```
    (all-the-icons-completion-mode))
```

8.3 Orderless

Allows for out-of-order pattern matching in minibuffer.

```
(use-package orderless
  :custom
  (completion-styles '(orderless basic))
  (completion-category-overrides '((file (styles partial-completion))))
  (completion-category-defaults nil) ;; Disable defaults, use our
  ↪  settings
  (completion-pcm-leading-wildcard t)) ;; Emacs 31: partial-completion
  ↪  behaves like substringing
```

8.4 Consult

Better searches with narrowing features (more than just compeltions).
#+begin$_{\text{sc}}$ emacs-lisp (use-package consult) #+end$_{\text{src}}$

8.5 Embark

Allows for the operation on completions in a minibuffer.

```
(use-package embark
  :bind
  (("C-." . embark-act)
   ("M-." . embark-dwim)
   ("C-h B" . embark-bindings))
  :custom
  (embark-quit-after-action nil)
  :config
  ;; Hide the mode line of the Embark live/completions buffers
  (add-to-list 'display-buffer-alist
               '("\\`\\*Embark Collect \\(Live\\|Completions\\)\\*"
                 nil
                 (window-parameters (mode-line-format . none))))

  )

;; Consult with Embark
(use-package embark-consult
  :hook
  (embark-collect-mode . consult-preview-at-point-mode))
```

8.6 Corfu

Basically company but in the vertico style.

```
(use-package corfu
  :custom
  (corfu-cycle t)
  (corfu-quit-at-boundary nil)
  (corfu-quit-no-match nil)
  :bind (("C-/" . completion-at-point)
         :map corfu-map
         ("C-j" . corfu-next)
         ("C-k" . corfu-previous))
  :init
  (global-corfu-mode)
  (corfu-history-mode))
```

9 Org Mode

9.1 Org Faces

```
(defun dasker/org-faces ()
  (set-face-attribute 'org-todo nil :height 0.7)
  (set-face-attribute 'org-done nil :height 0.7)
  (set-face-attribute 'org-headline-done nil :foreground nil)
  (set-face-attribute 'org-tag nil :height 0.7)
  (set-face-attribute 'org-priority nil :height 0.7)
  (set-face-attribute 'org-block-begin-line nil :height 0.7)
  (set-face-attribute 'org-block-end-line nil :height 0.7)
  (modus-themes-with-colors
   (set-face-attribute 'org-hide nil :foreground bg-main :background
   ↪  bg-main))

  (set-face-attribute 'org-level-1 nil :height 1.4)
  (set-face-attribute 'org-level-2 nil :height 1.3)
  (set-face-attribute 'org-level-3 nil :height 1.2)
  (set-face-attribute 'org-level-4 nil :height 1.1)
  (set-face-attribute 'org-level-5 nil :height 1.1)
  (set-face-attribute 'org-level-6 nil :height 1.1)
  (set-face-attribute 'org-level-7 nil :height 1.1)
  (set-face-attribute 'org-level-8 nil :height 1.1)
  (set-face-attribute 'org-block nil
                      :foreground nil :inherit '(fixed-pitch default))
  (set-face-attribute 'org-code nil :inherit '(shadow fixed-pitch
  ↪  default))
  (set-face-attribute 'org-verbatim nil
                      :inherit '(shadow fixed-pitch))
```

```
  (set-face-attribute 'org-special-keyword nil
                      :inherit '(font-lock-comment-face fixed-pitch))
  (set-face-attribute 'org-meta-line nil
                      :inherit '(font-lock-comment-face fixed-pitch))
  (set-face-attribute 'org-checkbox nil :inherit 'fixed-pitch)
  (set-face-attribute 'org-table nil :inherit '(shadow fixed-pitch))
  (set-face-attribute 'org-formula nil :inherit 'fixed-pitch)
  (set-face-attribute 'org-document-info-keyword nil
                      :inherit '(shadow fixed-pitch))
  (set-face-attribute 'org-document-title nil
                      :weight 'bold :height 1.8)
  (require 'org-indent)
  (set-face-attribute 'org-indent
                      nil :inherit '(org-hide fixed-pitch))
  (require 'org-faces)
  (set-face-attribute 'org-latex-and-related nil :inherit 'fixed-pitch)
  (setq org-todo-keyword-faces
        '(("PROGRESS" . (:inherit (warning org-todo)))
          ("WAITING" . (:inherit (shadow org-todo)))))
  )
```

9.2 Basic Setup

```
(defun dasker/org-mode-setup ()
  (org-indent-mode 1)
  (variable-pitch-mode 1)
  (auto-fill-mode 0)
  (visual-line-mode 1)
  (setq evil-auto-indent nil))

(use-package org
  :hook ((org-mode . dasker/org-mode-setup)
         ;; inhibits the auto pairing of '<' characters
         (org-mode . (lambda ()
                       (setq-local electric-pair-inhibit-predicate
                                   '(lambda (c)
                                      (if (char-equal c 60) t nil))))))
  :custom
  ;; Basic
  (undo-limit 50000)                    ;; lower undo-limit for
  ↪  performance
  (org-list-allow-alphabetical 1)       ;; allow alphabetical lists
  (org-startup-with-inline-images 1)    ;; show images on startup
  (org-hide-emphasis-markers t)         ;; hide the emphasis markers
  (org-hide-leading-stars t)           ;; hide the leading stars of
  ↪  heading
  (org-ellipsis " ")                    ;; folded indicator
```

```
(org-pretty-entities t)              ;; utf8 characters in entities
(org-startup-folded 'fold)           ;; start with evertyhing folded
(org-fold-catch-invisible-edits 'show-and-error)
(org-catch-invisible-edits 'show-and-error)
(org-special-ctrl-a/e t)
(org-insert-heading-respect-content t)

;; src blocks
(org-edit-src-content-indentation 0) ;; no spaces at start of src
↪  block
(org-src-fontify-natively t)         ;; use native fonts
(org-src-tab-acts-natively t)        ;; take tab back
(org-latex-src-block-backend 'engraved) ;; typesetting src blocks
↪  latex

;; Task
(org-log-done 'time)      ;; log time when done with a task
(org-log-into-drawer t)   ;; log into a drawer so I can hide it
(org-lowest-priority ?C)  ;; lowest prioirty for task is C
(org-default-priority ?B) ;; default priority is B
(org-todo-keywords        ;; adds different todo keywords
 '((sequence
    "TODO(t)" "PROGRESS(p)" "WAITING(w)"
    "|"
    "DONE(d)")))
(org-priority-faces
 '((?A . (:inherit (bold org-priority)))
   (?B . (:inherit (org-priority)))
   (?C . (:inherit (shadow org-priority)))))

;; Tags
(org-tags-column 0)     ;; put tags right after header
(org-auto-align-tags 1) ;; align tags automatically
(org-tag-alist '(("idea" . ?i)
                 ("research" . ?r)
                 (:startgroup . nil)
                 ("personal" . ?p)
                 ("work" . ?w)
                 (:endgroup . nil)))

;; Org Agenda
(org-agenda-start-with-log-mode t) ;; start agenda in log mode
(org-agenda-files                  ;; where to find agenda files
 '("~/orgfiles/refile.org"
   "~/orgfiles/ideas.org"
   "~/orgfiles/work.org"
   "~/orgfiles/personal.org"))
(org-agenda-start-on-weekday 0)    ;; start on sunday
```

```
  (org-agenda-window-setup 'current-window)

  ;; LaTeX
  (org-latex-compiler "lualatex")   ;; use lualatex to export
  (org-highlight-latex-and-related  ;; latex hl
   '(native latex script entities))

  ;; Capture
  (org-refile-targets
   '((org-agenda-files . (:level 1))))
  (org-refile-use-outline-path 'file)
  (org-capture-templates
   '(("t" "Tasks")
     ("tt" "Task" entry (file "~/orgfiles/refile.org")
      "* TODO %?\n %U\n %i" :empty-lines 1)
     ("tw" "Work Task" entry (file "~/orgfiles/work.org")
      "* TODO %? :work:\n %U\n %i" :empty-lines 1)
     ("tp" "Personal Task" entry (file "~/orgfiles/personal.org")
      "* TODO %? :personal:\n %U\n %i" :empty-lines 1)
     ("tr" "Research Task" entry (file "~/orgfiles/research.org")
      "* TODO %? :research:\n %U\n %i" :empty-lines 1)
     ("i" "Idea")
     ("ii" "Idea" entry (file "~/orgfiles/ideas.org")
      "* TODO %? :idea:\n %U\n %i" :empty-lines 1)
     ("ir" "Research Idea" entry (file "~/orgfiles/research.org")
      "* TODO %? :research:idea:\n %U\n %i" :empty-lines 1)))

  :config
  (dasker/org-faces)
  (add-hook 'modus-themes-after-load-theme-hook #'dasker/org-faces)
  (add-to-list 'org-link-frame-setup '(file . find-file))
  (require 'ox)
  (add-to-list 'org-latex-classes
               '("extarticle" "\\documentclass{extarticle}"
                 ("\\section{%s}" . "\\section*{%s}")
                 ("\\subsection{%s}" . "\\subsection*{%s}")
                 ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
                 ("\\paragraph{%s}" . "\\paragraph*{%s}")
                 ("\\subparagraph{%s}" . "\\subparagraph*{%s}")))
  (setq org-image-actual-width nil)
  (with-eval-after-load 'ox-html
    (setq org-html-htmlize-output-type nil))
  )
```

9.3 Org Auto Tangle

```
(use-package org-auto-tangle
  :load-path "site-lisp/org-auto-tangle/"    ;; this line is necessary
  ↪  only if you cloned the repo in your site-lisp directory
  :defer t
  :hook (org-src-mode . org-auto-tangle-mode))
```

9.4 Evil-Org

```
(use-package evil-org
  :after org
  :hook (org-mode . (lambda () evil-org-mode))
  :config
  (require 'evil-org-agenda)
  (evil-org-agenda-set-keys))
```

9.5 Citations

9.5.1 citar

```
(use-package citar
  :after all-the-icons
  :custom
  (org-cite-global-bibliography '("~/orgfiles/refs/Orged.bib"
                                  "~/Documents/cv/content/me.bib"))
  (org-cite-insert-processor 'citar)
  (org-cite-follow-processor 'citar)
  (org-cite-activate-processor 'citar)
  (citar-bibliography org-cite-global-bibliography)
  (citar-notes-paths '("~/orgfiles/refs/notes/"))
  :hook
  (LaTeX-mode . citar-capf-setup)
  (org-mode . citar-capf-setup)
  :config
  (defvar citar-indicator-files-icons
    (citar-indicator-create
     :symbol (all-the-icons-faicon
              "file-o"
              :face 'all-the-icons-green
              :v-adjust -0.1)
     :function #'citar-has-files
     :padding "  " ; need this because the default padding is too low for
     ↪  these icons
     :tag "has:files"))

  (defvar citar-indicator-links-icons
```

```
    (citar-indicator-create
     :symbol (all-the-icons-octicon
              "link"
              :face 'all-the-icons-orange
              :v-adjust 0.01)
     :function #'citar-has-links
     :padding "  "
     :tag "has:links"))

  (defvar citar-indicator-notes-icons
    (citar-indicator-create
     :symbol (all-the-icons-material
              "speaker_notes"
              :face 'all-the-icons-blue
              :v-adjust -0.3)
     :function #'citar-has-notes
     :padding "  "
     :tag "has:notes"))

  (defvar citar-indicator-cited-icons
    (citar-indicator-create
     :symbol (all-the-icons-faicon
              "circle-o"
              :face 'all-the-icons-green)
     :function #'citar-is-cited
     :padding "  "
     :tag "is:cited"))

  (setq citar-indicators
        (list citar-indicator-files-icons
              citar-indicator-links-icons
              citar-indicator-notes-icons
              citar-indicator-cited-icons)))

(use-package citar-embark
  :after (citar embark)
  :no-require
  :config (citar-embark-mode))
```

9.6 Visual Column Alignment

```
  ;; Center org 80 character line in buffer
  (defun dasker/org-mode-visual-fill ()
    (setq visual-fill-column-width 80
          visual-fill-column-center-text t)
    (visual-fill-column-mode 1))
  (use-package visual-fill-column
```

```
  :hook ((org-mode . dasker/org-mode-visual-fill)))
```

9.7 Org Modern

```
(use-package org-modern
  :hook ((org-mode . org-modern-mode)
         (org-agenda-finalize . org-modern-agenda))
  :config
  (set-face-attribute 'org-modern-symbol nil
                      :height robby-default-font-size)
  :custom
  (org-modern-keyword nil)
  (org-modern-priority nil)
  (org-modern-tag nil)
  (org-modern-timestamp nil)
  (org-modern-todo nil)
  (org-modern-table nil)
  (org-modern-block-name nil)
  (org-modern-block-fringe nil)

  ;; Agenda Styling
  (org-agenda-tags-column 0)
  (org-agenda-block-separator ?-))
```

9.8 Org Tempo

```
(with-eval-after-load 'org
  (require 'org-tempo)

  ;; Added one at a time
  (add-to-list 'org-structure-template-alist
               '("el" . "src emacs-lisp"))
  (add-to-list 'org-structure-template-alist
               '("jv" . "src java")))
```

9.9 Org Code Listings

```
(use-package engrave-faces
  :after org)
```

9.10 Org Babel Languages Configuration

```
(with-eval-after-load 'org
  (setq org-src-preserve-indentation t)
  (org-babel-do-load-languages
   'org-babel-load-languages
   '((emacs-lisp . t)
     (java . t)
     (latex . t)
     (python . t)
     (makefile . t)
     (C . t)))
  (setq org-confirm-babel-evaluate nil)
  (setq org-babel-python-command "python3"))
```

10 Terminals and Shells

10.1 vterm

```
(use-package vterm
  :defer t
  :commands (vterm
             vterm-other-window
             projectile-run-vterm
             projectile-run-vterm-other-window))
```

10.2 Vscode Terminal Buffer

10.2.1 Function

Launch vscode like terminal in vertically split buffer

```
(defun launch-xterm ()
  (interactive)
  (let ((buf (get-buffer "vscode-vterm-window")))
    (if (and buf (get-buffer-window buf))
        ;; If it's visible, hide it
        (delete-windows-on buf)
      ;; If it's hidden or doesn't exist, display it
      (let ((new-win (split-window-below)))
        (select-window new-win)
        (if buf
            (switch-to-buffer buf)
          (vterm "vscode-vterm-window"))))))
```

10.2.2 Keybind

Open Vscode terminal in vertical buffer with C-‘.

```
(global-set-key (kbd "C-`") 'launch-xterm)
```